

# COMP3131/9102: Programming Languages and Compilers

*Jingling Xue*

School of Computer Science and Engineering  
The University of New South Wales  
Sydney, NSW 2052, Australia

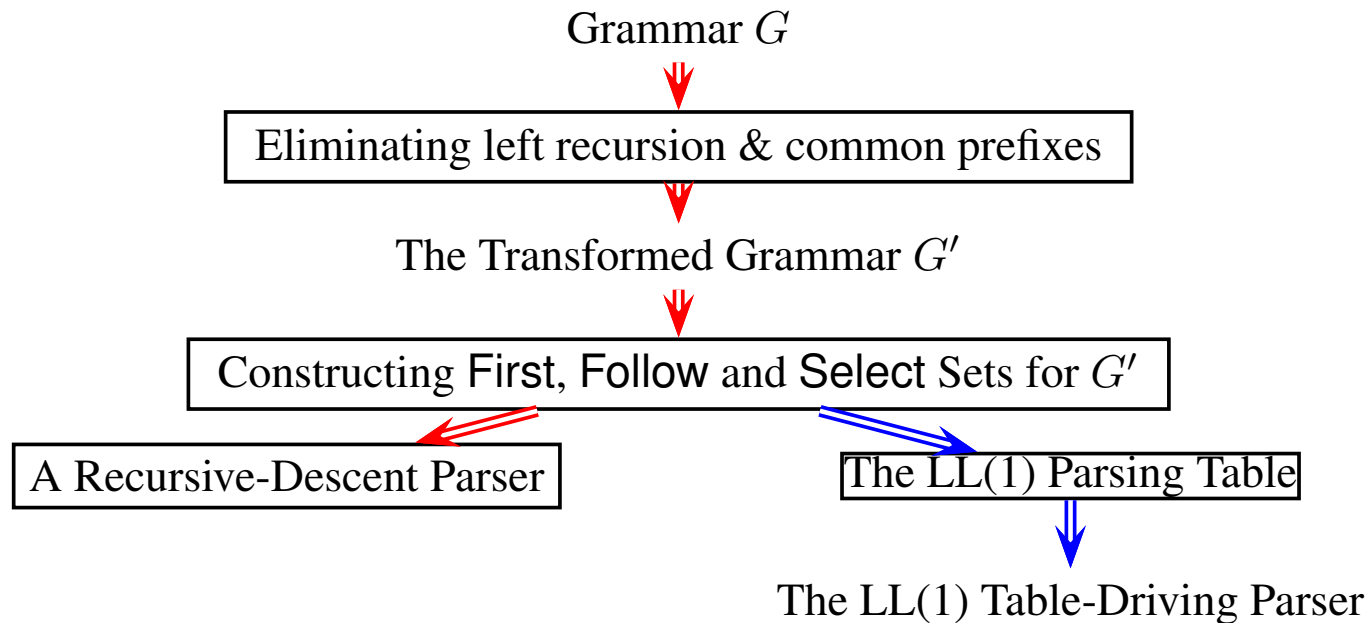
<http://www.cse.unsw.edu.au/~cs3131>

<http://www.cse.unsw.edu.au/~cs9102>

Copyright ©2024, Jingling Xue

## Week 9 (2nd Lecture): Table-Driven Top-Down Parsing

1. LL(1) table-driven parsing
2. Parser generators
3. Error recovery



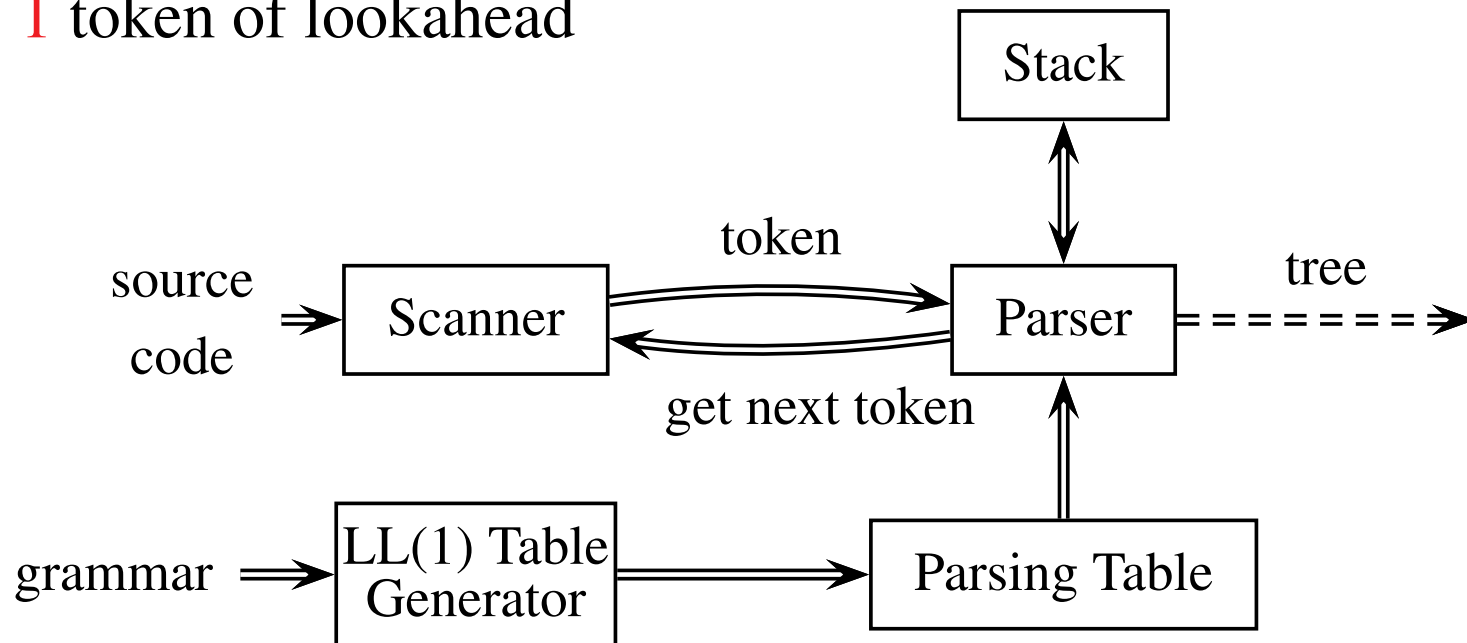
- The red parts done last week
- The the blue parts today

## Predictive Non-Recursive Top-Down Parsers

- **Recursion = Iteration + Stack**
- Recursive calls in a recursive-descent parser can be implemented using
  - an explicit stack, and
  - a parsing table
- Understanding one helps your understanding the other

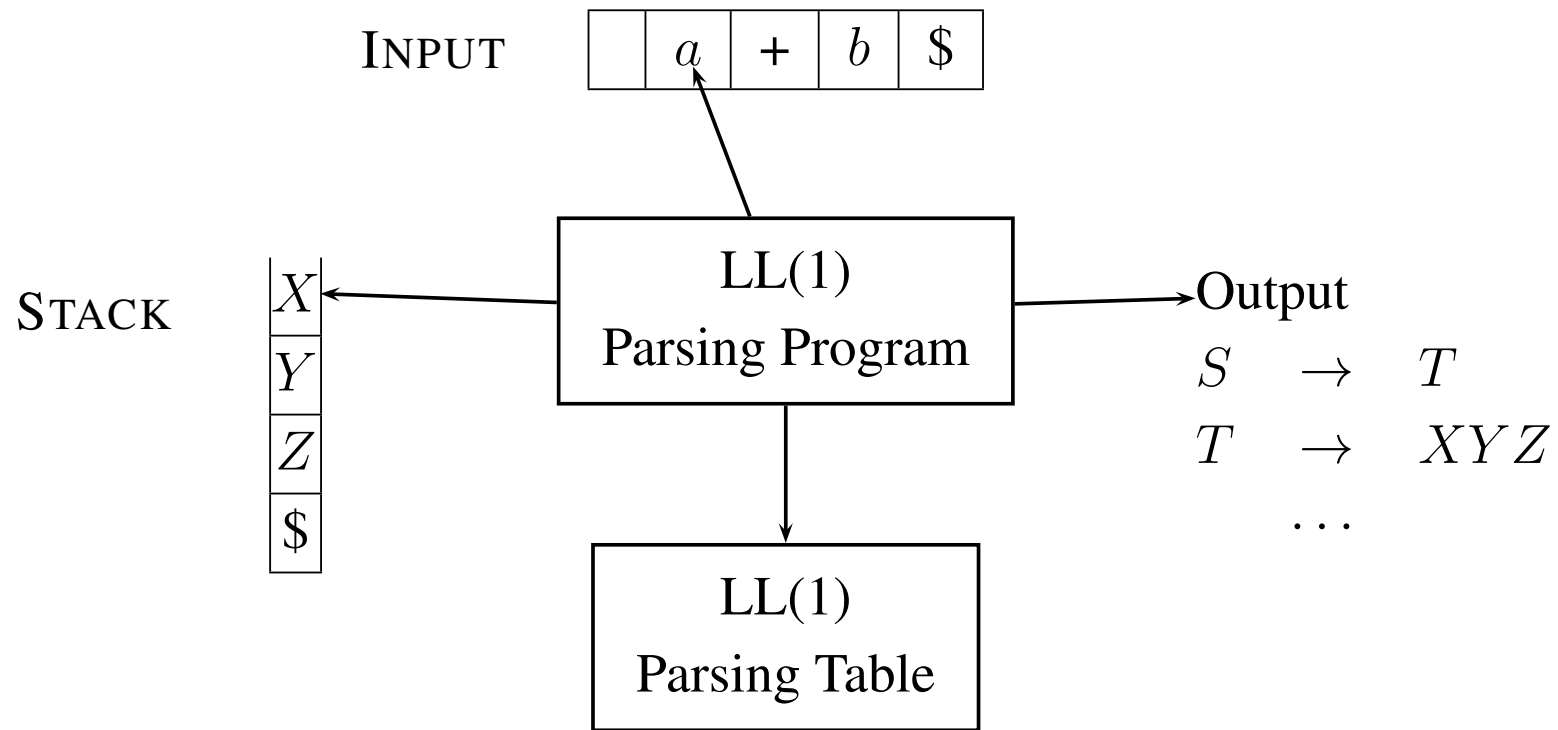
## The Structure of a Table-Driven LL(1) Parser

- Input parsed from **l**eft to right
- **L**eftmost derivation
- **1** token of lookahead



- LR(1) parsers (almost always table-driven) also built this way

## The Model of an LL(1) Table-Driven Parser



Output:

- The productions used (representing the leftmost derivation), or
- An AST (**built by a parser (Assignment 3)**)

## The LL(1) Parsing Program

Push \$ onto the stack

Push the start symbol onto the stack

**WHILE** (stack not empty) **DO**

**BEGIN**

Let  $X$  be the top stack symbol and  $a$  be the lookahead symbol in the input

**IF**  $X$  is a terminal **THEN**

**IF**  $X = a$  then pop  $X$  and get the next token */\* match \*/*

**ELSE** error

**ELSE** */\*  $X$  is a nonterminal \*/*

**IF**  $Table[X, a]$  nonblank **THEN**

Pop  $X$

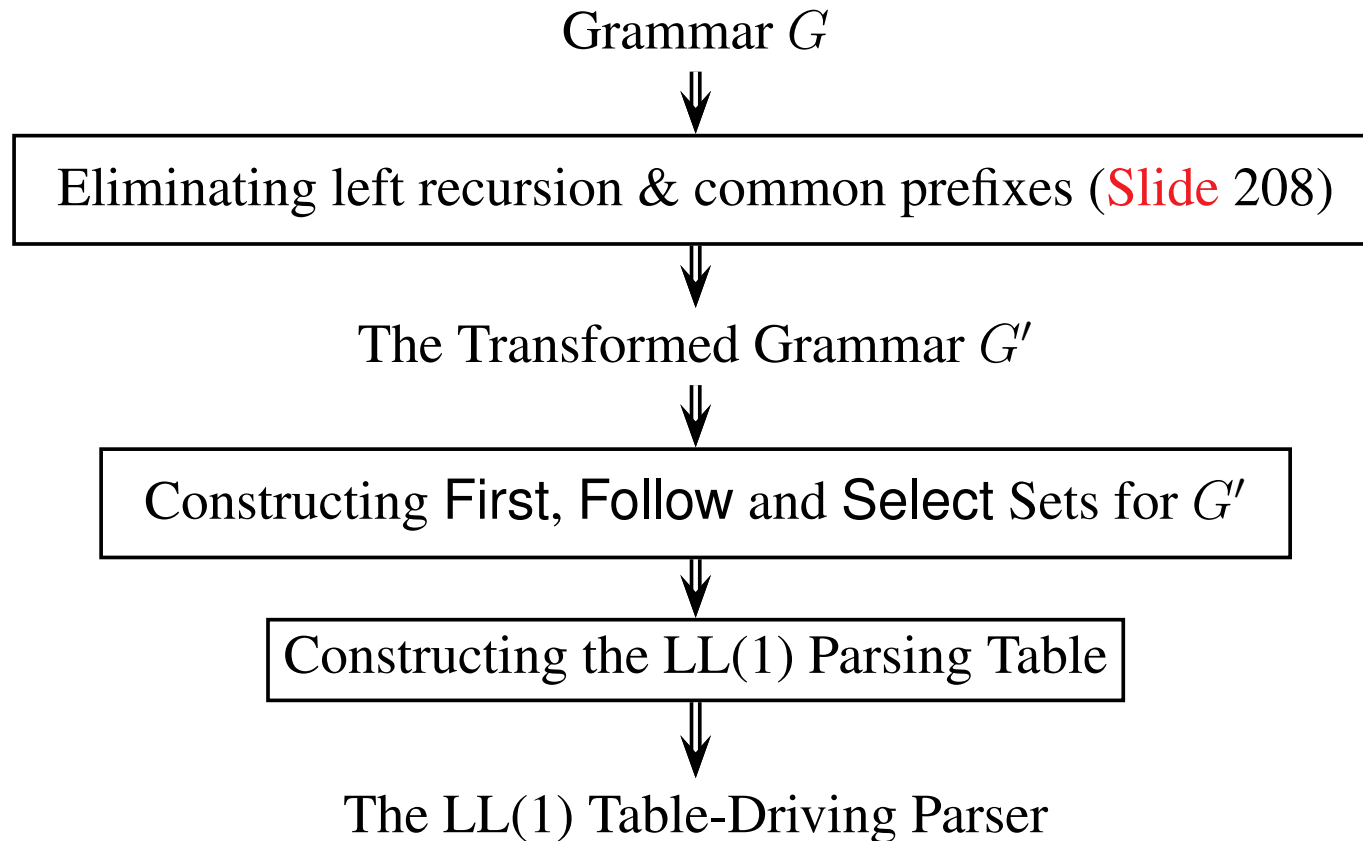
Push  $Table[X, a]$  onto stack in the *reverse order*

**ELSE** error

**END**

The parsing is successful when the stack is empty and no errors reported

## Building a Table-Driving Parser from a Grammar



- Follow Slide 208 to eliminate left recursion to get a BNF grammar
- Can build a table-driven parser for EBNF as well (but not considered)

## The Expression Grammar

- The grammar with left recursion:

**Grammar 1:**  $E \rightarrow E + T \mid E - T \mid T$   
 $T \rightarrow T * F \mid T / F \mid F$   
 $F \rightarrow \mathbf{INT} \mid (E)$

- The transformed grammar **without** left recursion:

**Grammar 2:**  $E \rightarrow TQ$   
 $Q \rightarrow +TQ \mid -TQ \mid \epsilon$   
 $T \rightarrow FR$   
 $R \rightarrow *FR \mid /FR \mid \epsilon$   
 $F \rightarrow \mathbf{INT} \mid (E)$



## First and Follow Sets for Grammar 2

- First sets:

$$\begin{aligned}
 \text{First}(TQ) = \text{First}(FR) &= \{ (, i \\
 \text{First}(Q) &= \{ +, -, \epsilon \} \\
 \text{First}(R) &= \{ *, /, \epsilon \} \\
 \text{First}(+TQ) &= \{ + \} \\
 \text{First}(-TQ) &= \{ - \} \\
 \text{First}(*FR) &= \{ * \} \\
 \text{First}(/FR) &= \{ / \} \\
 \text{First}((E)) &= \{ ( \} \\
 \text{First}(i) &= \{ i \}
 \end{aligned}$$

- Follow sets:

$$\begin{aligned}
 \text{Follow}(E) &= \{ \$, ) \} \\
 \text{Follow}(Q) &= \{ \$, ) \} \\
 \text{Follow}(T) &= \{ +, -, \$, ) \} \\
 \text{Follow}(R) &= \{ +, -, \$, ) \} \\
 \text{Follow}(F) &= \{ +, -, *, /, \$, ) \}
 \end{aligned}$$

## Select Sets for Grammar 2

$$\begin{aligned}
 \text{Select}(E \rightarrow TQ) &= \text{First}(TQ) &&= \{ (, \mathbf{INT} \} \\
 \text{Select}(Q \rightarrow + TQ) &= \text{First}(+TQ) &&= \{ + \} \\
 \text{Select}(Q \rightarrow - TQ) &= \text{First}(-TQ) &&= \{ - \} \\
 \text{Select}(Q \rightarrow \epsilon) &= (\text{First}(\epsilon) - \{ \epsilon \}) \cup \text{Follow}(Q) &&= \{ ), \$ \} \\
 \text{Select}(T \rightarrow FR) &= \text{First}(FR) &&= \{ (, \mathbf{INT} \} \\
 \text{Select}(R \rightarrow * FR) &= \text{First}(*FR) &&= \{ * \} \\
 \text{Select}(R \rightarrow / FR) &= \text{First}(/FR) &&= \{ / \} \\
 \text{Select}(R \rightarrow \epsilon) &= (\text{First}(\epsilon) - \{ \epsilon \}) \cup \text{Follow}(T) &&= \{ +, -, ), \$ \} \\
 \text{Select}(F \rightarrow \mathbf{INT}) &= \text{First}(\mathbf{INT}) &&= \{ \mathbf{INT} \} \\
 \text{Select}(F \rightarrow (E)) &= \text{First}((E)) &&= \{ ( \}
 \end{aligned}$$

## The Rules for Constructing an LL(1) Parsing Table

For every production of the  $A \rightarrow \alpha$  in the grammar, do:

for all  $a$  in **Select**( $A \rightarrow \alpha$ ), set  $Table[A, a] = \alpha$

## LL(1) Parsing Table for Grammar 2

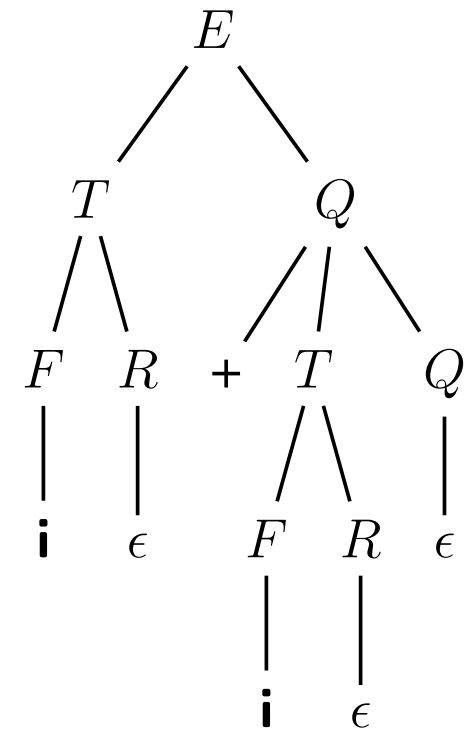
|          | <b>INT</b> | <b>+</b>   | <b>−</b>   | <b>*</b>   | <b>/</b>   | <b>(</b>   | <b>)</b> | <b>\$</b> |
|----------|------------|------------|------------|------------|------------|------------|----------|-----------|
| <i>E</i> | <i>TQ</i>  |            |            |            |            | <i>TQ</i>  |          |           |
| <i>Q</i> |            | <i>+TQ</i> | <i>−TQ</i> |            |            |            | €        | €         |
| <i>T</i> | <i>FR</i>  |            |            |            |            | <i>FR</i>  |          |           |
| <i>R</i> |            | €          | €          | <i>*FR</i> | <i>/FR</i> |            | €        | €         |
| <i>F</i> | <b>INT</b> |            |            |            |            | <i>(E)</i> |          |           |

- A 2D tabular representation of the **Select** sets
- The blanks are errors.

# An LL(1) Parse on Input $i+i$ : $INT \iff i$

| STACK   | INPUT   | PRODUCTION               | DERIVATION                 |
|---------|---------|--------------------------|----------------------------|
| $\$E$   | $i+i\$$ | $E \rightarrow TQ$       | $E \Rightarrow_{lm} TQ$    |
| $\$QT$  | $i+i\$$ | $T \rightarrow FR$       | $\Rightarrow_{lm} FRQ$     |
| $\$QRF$ | $i+i\$$ | $F \rightarrow i$        | $\Rightarrow_{lm} iRQ$     |
| $\$QRi$ | $i+i\$$ | pop and go to next token |                            |
| $\$QR$  | $+i\$$  | $R \rightarrow \epsilon$ | $\Rightarrow_{lm} iQ$      |
| $\$Q$   | $+i\$$  | $Q \rightarrow + TQ$     | $\Rightarrow_{lm} i + TQ$  |
| $\$QT+$ | $+i\$$  | pop and go to next token |                            |
| $\$QT$  | $i\$$   | $T \rightarrow FR$       | $\Rightarrow_{lm} i + FRQ$ |
| $\$QRF$ | $i\$$   | $F \rightarrow i$        | $\Rightarrow_{lm} i + iRQ$ |
| $\$QRi$ | $i\$$   | pop and go to next token |                            |
| $\$QR$  | $\$$    | $R \rightarrow \epsilon$ | $\Rightarrow_{lm} i + iRQ$ |
| $\$Q$   | $\$$    | $Q \rightarrow \epsilon$ | $\Rightarrow_{lm} i + iQ$  |
| $\$$    | $\$$    |                          |                            |

PARSE TREE



## An LL(1) Parse on an Erroneous Input “()”

| STACK     | INPUT  | PRODUCTION                               | DERIVATION                        |
|-----------|--------|------------------------------------------|-----------------------------------|
| $\$E$     | $()\$$ | $E \rightarrow TQ$                       | $E \Rightarrow_{\text{lm}} TQ$    |
| $\$QT$    | $()\$$ | $T \rightarrow FR$                       | $E \Rightarrow_{\text{lm}} FRQ$   |
| $\$QRF$   | $()\$$ | $F \rightarrow (E)$                      | $E \Rightarrow_{\text{lm}} (E)RQ$ |
| $\$QR)E($ | $()\$$ | pop and go to next token                 |                                   |
| $\$QR)E$  | $)\$$  | * * * Error: no table entry for $[E, )]$ |                                   |

A better error message: **expression missing inside ( )**

## LL(1) Grammars and Table-Driven LL(1) Parsers

- Like recursive descent, table-driven LL(1) parsers can only parse LL(1) grammars. Conversely, only LL(1) grammars can be parsed by the table-driven LL(1) parsers.
- Definition of LL(1) grammar given in Slide 189
- Definition of LL(1) grammar – using the parsing table:  
A grammar is LL(1) if every table entry contains at most one production.

## Why Table-Driven LL(1) Parsers Cannot Handle Left Recursions?

- A grammar with left recursion:

$$\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \text{id} \mid \text{id}$$

- Select Sets:

$$\text{Select}(\langle \text{expr} \rangle + \text{id}) = \{\text{id}\}$$

$$\text{Select}(\text{id}) = \{\text{id}\}$$

- The parsing table:

|                               | <b>id</b>                                              | \$ |
|-------------------------------|--------------------------------------------------------|----|
| $\langle \text{expr} \rangle$ | $\langle \text{expr} \rangle + \text{id}$<br><b>id</b> |    |

*Table*[ $\langle \text{expr} \rangle$ , **id**] contains two entries!

- Any grammar with left recursions is not LL(1)



## Why Table-Driven LL(1) Parsers Cannot Handle Left Recursions (Cont'd)?

- Eliminating the left recursion yields an LL(1) grammar:

$$\begin{aligned}\langle \text{expr} \rangle &\rightarrow \mathbf{id} \langle \text{expr-tail} \rangle \\ \langle \text{expr-tail} \rangle &\rightarrow \epsilon \mid + \mathbf{id} \langle \text{expr-tail} \rangle\end{aligned}$$

- Select Sets:

$$\text{Select}(\langle \text{expr} \rangle \rightarrow \mathbf{id} \langle \text{expr-tail} \rangle) = \{\mathbf{id}\}$$

$$\text{Select}(\langle \text{expr-tail} \rangle \rightarrow \epsilon) = \{\$ \}$$

$$\text{Select}(\langle \text{expr-tail} \rangle \rightarrow + \mathbf{id} \langle \text{expr-tail} \rangle) = \{+\}$$

- The parsing table for the transformed grammar:

|                                    | <b>id</b>                                    | +                                                     | \$         |
|------------------------------------|----------------------------------------------|-------------------------------------------------------|------------|
| $\langle \text{expr} \rangle$      | <b>id</b> $\langle \text{expr-tail} \rangle$ |                                                       |            |
| $\langle \text{expr-tail} \rangle$ |                                              | <b>+</b> <b>id</b> $\langle \text{expr-tail} \rangle$ | $\epsilon$ |

## Why LL(1) Table-Driven Parsers Cannot Handle Common Prefixes?

- A grammar with a common prefix:

$$\begin{aligned} S &\rightarrow \mathbf{if} (E) S \mid \mathbf{if} (E) S \mathbf{else} S \mid \mathbf{s} \\ E &\rightarrow e \end{aligned}$$

- Select sets:

$$\begin{aligned} \text{Select}(S \rightarrow \mathbf{if} (E) S) &= \{\mathbf{if}\} \\ \text{Select}(S \rightarrow \mathbf{if} (E) S \mathbf{else} S) &= \{\mathbf{if}\} \end{aligned}$$

- Any grammar with common prefixes is not LL(1)
- Eliminating the common prefix **does not** yield an LL(1) grammar:

$$\begin{aligned} S &\rightarrow \mathbf{if} (E) SQ \mid \mathbf{s} \\ Q &\rightarrow \mathbf{else} S \mid \epsilon \\ E &\rightarrow e \end{aligned}$$

## Why LL(1) Table-Driven Parsers Cannot Handle Common Prefixes (Cont'd)?

- Select sets:

$$\begin{aligned}
 \text{Select}(S \rightarrow \mathbf{if}(E)SQ) &= \{\mathbf{if}\} \\
 \text{Select}(S \rightarrow s) &= \{s\} \\
 \text{Select}(Q \rightarrow \mathbf{else}S) &= \{\mathbf{else}\} \\
 \text{Select}(Q \rightarrow \epsilon) &= \{\mathbf{else}, \epsilon\} \\
 \text{Select}(E \rightarrow e) &= \{e\}
 \end{aligned}$$

- The parsing table:

|          | <b>if</b>                                | ( | e        | ) | s        | <b>else</b>                        | \$         |
|----------|------------------------------------------|---|----------|---|----------|------------------------------------|------------|
| <i>S</i> | <b>if</b> <i>E</i> <b>then</b> <i>SQ</i> |   |          |   | <i>s</i> |                                    |            |
| <i>E</i> |                                          |   | <i>e</i> |   |          |                                    |            |
| <i>Q</i> |                                          |   |          |   |          | <b>else</b> <i>S</i><br>$\epsilon$ | $\epsilon$ |

- This modified grammar, although having no common prefixes, is still ambiguous.  
 You are referred to [Week 6 Tutorial](#). To resolve the ambiguity in the grammar, we make the convention to select **else** *S* as the table entry. This effectively implements the following rule:

Match an **else** to the most recent unmatched **then**

## Recognise Palindromes Easily

- Grammar:

$$S \rightarrow (S) \mid \epsilon$$

- Parsing Table:

|     |       |            |            |
|-----|-------|------------|------------|
|     | (     | )          | \$         |
| $S$ | $(S)$ | $\epsilon$ | $\epsilon$ |

- Try to parse the following three inputs:
  - $(( ))$
  - $(( )$
  - $( ) )$
- Cannot design a DFA/NFA to recognise the language  $L(S)$

## Week 10 (Mon): Table-Driven Top-Down Parsing

1. LL(1) table-driven parsing ✓
2. Parser generators

## The Expression Grammar

- The grammar with left recursion:

**Grammar 1:**  $E \rightarrow E + T \mid E - T \mid T$   
 $T \rightarrow T * F \mid T / F \mid F$   
 $F \rightarrow \text{INT} \mid (E)$

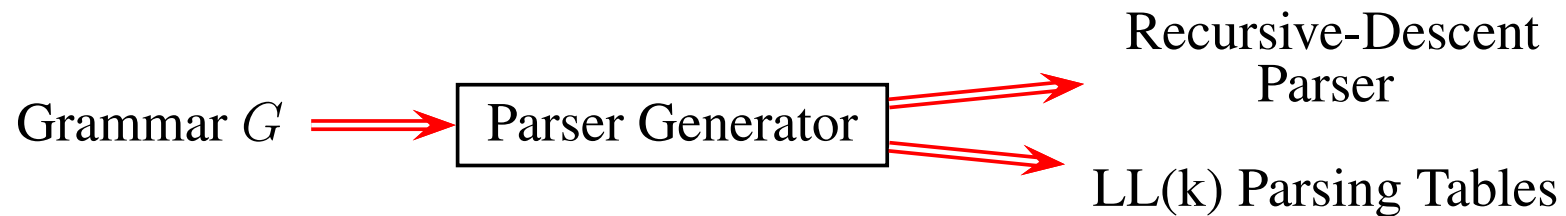
- Eliminating left recursion using the Kleene Closure

**Grammar 3:**  $E \rightarrow T ( "+" T \mid "-" T )^*$   
 $T \rightarrow F ( "*" F \mid "/" F )^*$   
 $F \rightarrow \text{INT} \mid "(" E ")"$

All tokens are enclosed in double quotes to distinguish them for the regular operators: (, ) and \*

- Compare with Slide 573

## Parser Generators (Generating Top-Down Parsers)



| Tool   | Grammar          | Accepted Parsers and Their Implementation Languages        |
|--------|------------------|------------------------------------------------------------|
| JavaCC | EBNF             | Recursive-Descent LL(1) (with some LL(k) portions) in Java |
| COCO/R | EBNF             | Recursive-Descent LL(1) in Pascal, C, C++, Java, etc.      |
| ANTLR  | Predicated LL(k) | Recursive-Descent LL(k) in C, C++, Java                    |

- These and other tools can be found on the internet
- **Predicated**: a conditional evaluated by the parser at run time to determine which of the two conflicting productions to use

$$Q \rightarrow \boxed{\text{if (lookahead is "else")}} \text{ else } S \mid \epsilon$$

where the condition inside the box resolves the dangling-else problem.

## Parser Generators (Generating Bottom-Up Parsers)



| Tool    | Grammar Accepted Parsers and Their Implementation Languages |                              |
|---------|-------------------------------------------------------------|------------------------------|
| Yacc    | BNF                                                         | LALR(1) table-driven in C    |
| JavaCUP | BNF                                                         | LALR(1) table-driven in Java |

- These and other tools can be found on the internet
- Will not deal with LR parsing in this course



## The JavaCC Spec for Grammar 3

```
/*
 * Parser.jj
 *
 * The scanner and parser for Grammar 3
 *
 * Install JavaCC from https://javacc.dev.java.net/
 *
 * 1. javacc Parser.jj
 * 2. javac Parser.java
 * 3. java Parser
 */

options {
  LOOKAHEAD=1;
}

PARSER_BEGIN(Parser)

public class Parser {

    public static void main(String args[]) throws ParseException {
        Parser parser = new Parser (System.in);
        while (true) {
            System.out.print("Enter Expression: ");
            System.out.flush();
            try {
                switch (parser.one_line()) {
```

```

        case -1:
            System.exit(0);
        default:
            System.out.println("Compilation was successful.");
            break;
    }
} catch (ParseException x) {
    System.out.println("Exiting.");
    throw x;
}
}

}

}

PARSER_END(Parser)

SKIP :
{
    " "
|    "\r"
|    "\t"
}

TOKEN :
{
    < EOL: "\n" >
}

TOKEN : /* OPERATORS */
{

```

```

        < PLUS: "+" >
|      < MINUS: "-" >
|      < MULTIPLY: "*" >
|      < DIVIDE: "/" >
}

TOKEN :
{
    < CONSTANT: ( <DIGIT> )+ >
|  < #DIGIT: ["0" - "9"] >
}

int one_line() :
{}
{
    Expr() <EOL>
        { return 1; }
| <EOL>
        { return 0; }
| <EOF>
        { return -1; }
}

void Expr() :
{ }
{
    Term() (( <PLUS> | <MINUS> ) Term())*
}

```

```
void Term() :  
{ }  
{  
    Factor() (( <MULTIPLY> | <DIVIDE> ) Factor())*  
}  
  
void Factor() :  
{ }  
{  
    <CONSTANT>  
    | "(" Term() ")"  
}
```

## Lecture 5: Top-Down Parsing: Table-Driven

1. LL(1) table-driven parsing ✓
2. Parser generators ✓

## Reading

- Table-driven parsing: pages 186 – 192 of Red Dragon or §4.4.4 of Purple Dragon
- JavaCC, ANTLR and COCO/R: available on the Web
- JavaCUP also available on the Web  
<http://www.cs.princeton.edu/~appel/modern/java/CUP/>
- Error recovery for LL parsers:
  - Using acceptance sets:
    - \* Pages 192 – 195 of Red Dragon / Pages §4.4.5 of Purple Dragon
    - \* [http://teaching.idallen.com/cst8152/98w/panic\\_mode.html](http://teaching.idallen.com/cst8152/98w/panic_mode.html)
  - Using continuation (pages 136 – 142 of Grune et al's 2000 compiler book. ISBN: 0-471-97697-0)

Last lecture: JVM + Revision